Full Stack Development with MERN

BookNest: Where Stories Nestle

Team ID: LTVIP2026TMIDS88565

Team Size: 4

Team Members and Roles

Name	Role
Chappidi babu	Team Leader / Backend Architect
Dhileswara Rao	Member / Frontend Developer
Kavyasri Bokka	Member / Full Stack Integration
Sagar debnath	Member / Database & Testing

1 Introduction

Project Title: ShopEZ

1. BookNest is a comprehensive online bookstore platform developed to simplify and enhance the digital book purchasing experience. Built using the modern MERN stack (MongoDB, Express.js, React.js, and Node.js), the application ensures high performance, scalability, and a smooth user interface for both readers and administrators.

2. Project Overview

Purpose

The purpose of BookNest is to provide a user-friendly, one-stop platform for discovering and purchasing books online. The project aims to simplify the book-buying experience by offering a centralized store where users can explore various genres such as fiction, non-fiction, academic, self-development, and more in one place.
Instead of browsing multiple websites, readers can easily search, view detailed book information, manage wishlists, and make secure purchases within a single platform.

Features

Book Catalog: Dynamic browsing of books with high-quality cover images, detailed descriptions, author information, and pricing.
• **Smart Filtering & Search:** Filter books by categories such as Fiction, Non-Fiction, Academic, Self-Development, and more. Users can also sort by price, ratings, or newest arrivals.
• **Wishlist & Cart Management:** Add books to wishlist or cart with real-time updates and easy management of selected items.
• **Responsive Design:** Fully optimized for seamless usage across Mobile, Tablet, and Desktop devices.
• **Admin Dashboard:** Complete CRUD (Create, Read, Update, Delete) functionality to manage books, categories, pricing, and inventory efficiently.
• **Secure User Authentication:** User registration and login system with secure authentication for personalized browsing and order management.
• **RESTful API Integration:** Smooth backend communication using APIs for dynamic data handling and performance optimation.

3. Architecture

Frontend (React.js)

3.1 The frontend of BookNest is architected as a Single Page Application (SPA) using React.js, ensuring fast navigation and a smooth user experience without full-page reloads. The application follows a modular component-based architecture, promoting reusability and maintainability.

3.2Key components such as Navbar, BookCard, FilterBar, and Wishlist/Cart components are reused across different pages to maintain UI consistency and clean design structure.

Backend (Node.js & Express.js)

The backend follows a RESTful API pattern. Built on Node.js, it uses Express to handle routing and middleware functionality. The server acts as a secure layer that validates user requests and coordinates data transactions between the client and the MongoDB database.

Database (MongoDB)

A NoSQL approach is used to store product and category data. This allows for flexible attributes across different product types. Mongoose is utilized as the ODM (Object Data Modeling) tool to enforce data structure and facilitate easy interaction with the database.

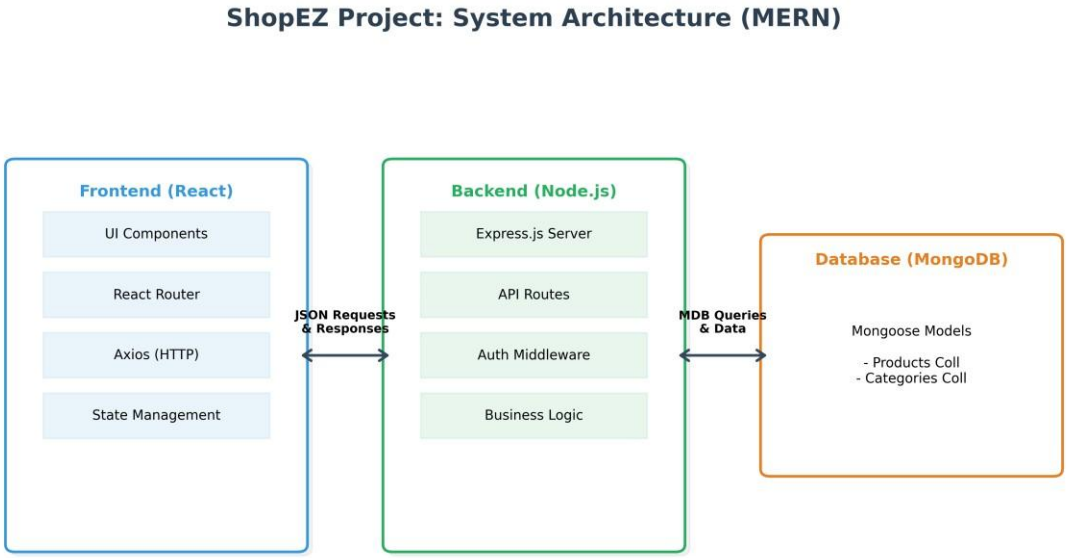

fig: Booknest architecture

4. Setup Instructions

Prerequisites

- Ensure the following are installed on your system:
- Node.js (v14 or higher)
- npm (Node Package Manager)
- MongoDB (Local Community Edition or Atlas URI)
- Git for version control

Installation Steps

1. Clone the repository: git https://github.com/kavyasri777/Apsche-mern-project

2. Setup Backend:

```
cd  server
npm install
```

Create a .env file and add your MONGO_URI and PORT (6001).

3. Setup Frontend:

```
cd  ../client
npm install
```

5. Folder Structure

Client (React)

```
client/
  ├── public/          # Static files
  ├── src/
  │   ├── components/    # UI Components (Navbar, Cards)
  │   ├── pages/         # Specific Views (Home, Shop, Admin)
  │   ├── App.js         # Routing and Main Logic
  │   └── index.js       # Entry point
  └── package.json       # Dependencies
```

Server (Node/Express)

```
  server/
  ├── .env             # Config environment variables
  ├── Schema.js        # Mongoose Data Models
  ├── index.js         # Server entry and API routes
└── package.json       # Dependencies
```

6. Running the Application

To start the application locally, run commands in separate terminals:

Frontend

Command: npm start (inside /client directory)

URL: http://localhost:3000

Backend

Command: npm start (inside /server directory)

URL: http://localhost:6001

7. API Documentation

Method	Endpoint	Description
GET	/fetch-categories	Retrieve all categories for filters
POST	/add-category	Admin: Create a new category
DELETE	/delete-category/:cat	Admin: Path-based deletion of category
GET	/fetch-products	Retrieve list of all shop items
POST	/add-product	Admin: Add a new inventory item
PUT	/update-product/:id	Admin: Update details for an existing item
DELETE	/delete-product/:id	Admin: Remove product by DB ID

8. Authentication

8.1 Authentication Mechanism ShopEZ employs a secure, credential-based authentication system to verify user identity.

- **Password Hashing:** To ensure data privacy, user passwords are never stored in plain text. The system utilizes **Bcrypt.js** for salting and hashing passwords before they are committed to the MongoDB database.
- **JWT Implementation:** Upon successful login, the server generates a **JSON Web Token (JWT)**. This token act as a "digital passport," allowing the frontend to make authenticated requests to protected API endpoints without needing to resend credentials.

8.2 Role-Based Access Control (RBAC) The application distinguishes between different user levels to maintain system integrity:

- **User Role:** Can browse products, manage their personal cart, and view their specific order history.
- **Admin Role:** Granted elevated privileges to access the Product Management dashboard, modify inventory, and manage global categories.

8.3 Session Management

- **Persistence:** The user's authentication state is managed using the **React Context API** or **Local Storage**. This ensures that the user remains logged in even after refreshing the browser page.
- **Logout Logic:** On logout, the client-side token is destroyed, and the global user state is cleared, preventing any further access to private routes.

8.4 Backend Security Middleware The Node.js server acts as the final gatekeeper. Every request to an admin-only endpoint (like POST /add-product) passes through a custom middleware function. This middleware:

1. Extracts the token from the request header.
2. Verifies the token's validity.
3. Checks the is Admin boolean within the decoded payload.
4. If unauthorized, it returns a 403 Forbidden or 401 Unauthorized status code.

9. User Interface

The visual identity of BookNest is designed to be clean, modern, and high-contrast.

Login page :

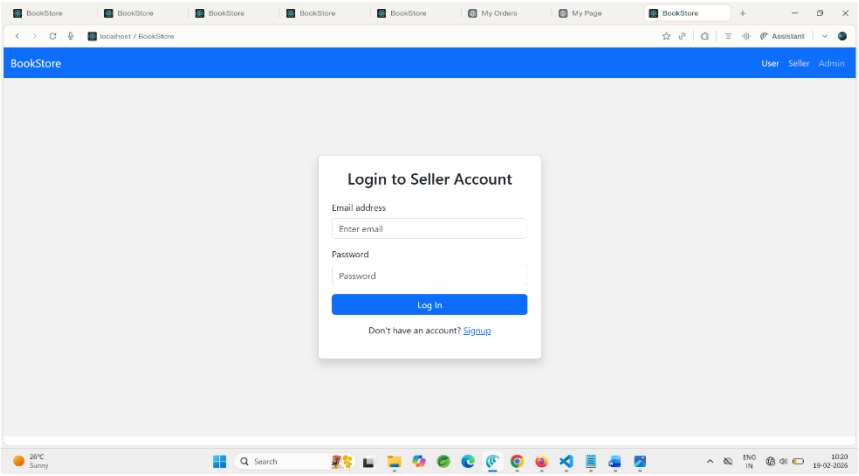

fig: Login with credentials

Home Page:

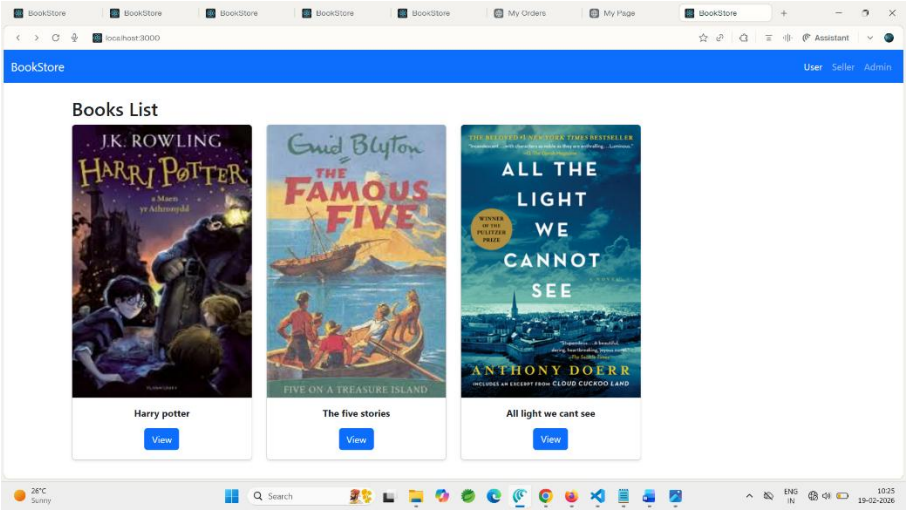

fig: Home page

User Wishlist:

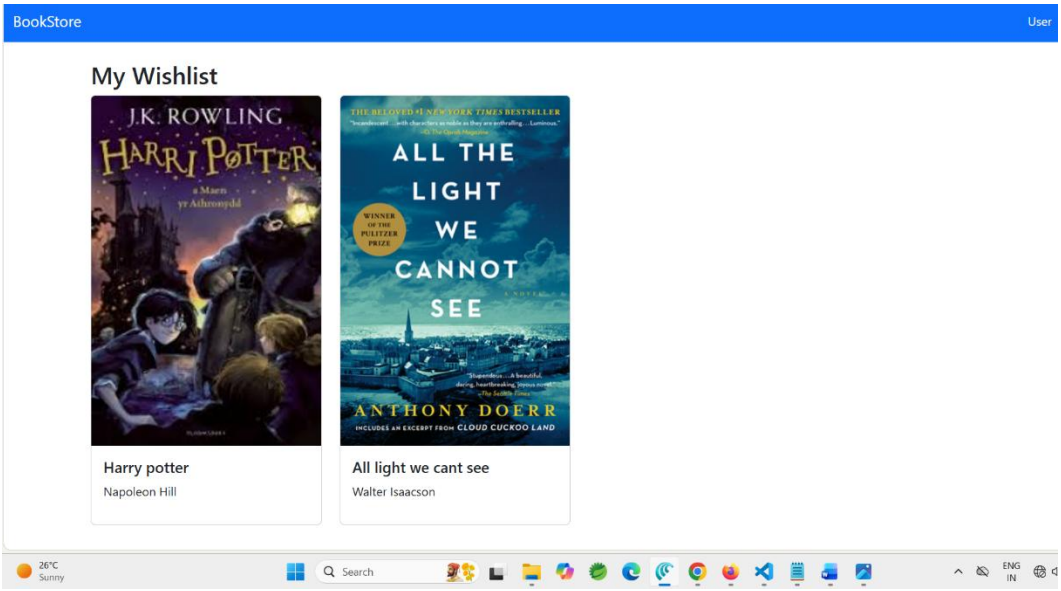

fig: User wishlist

User orders:

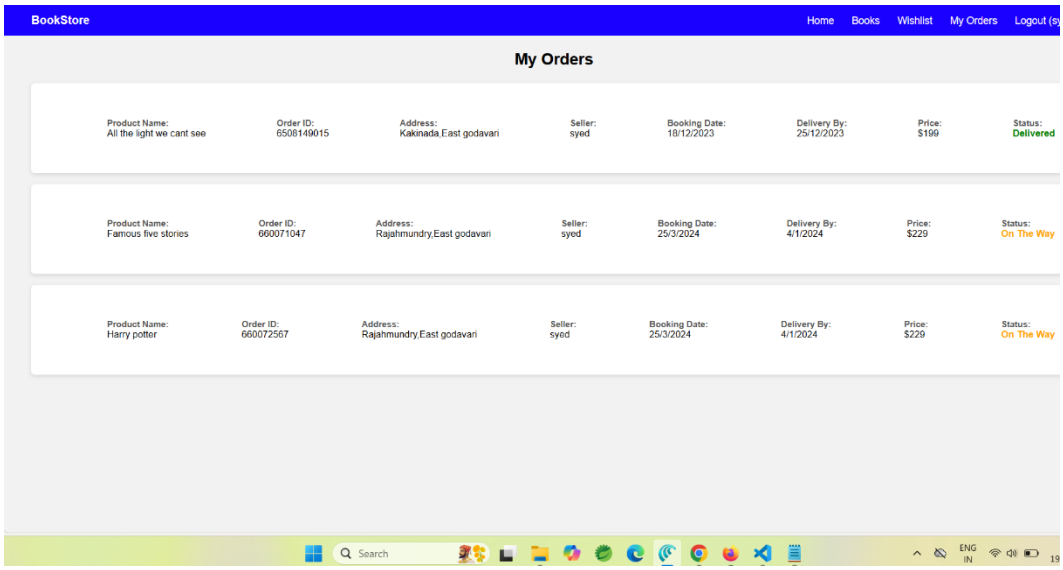

fig: Orders

Seller Dashboard:

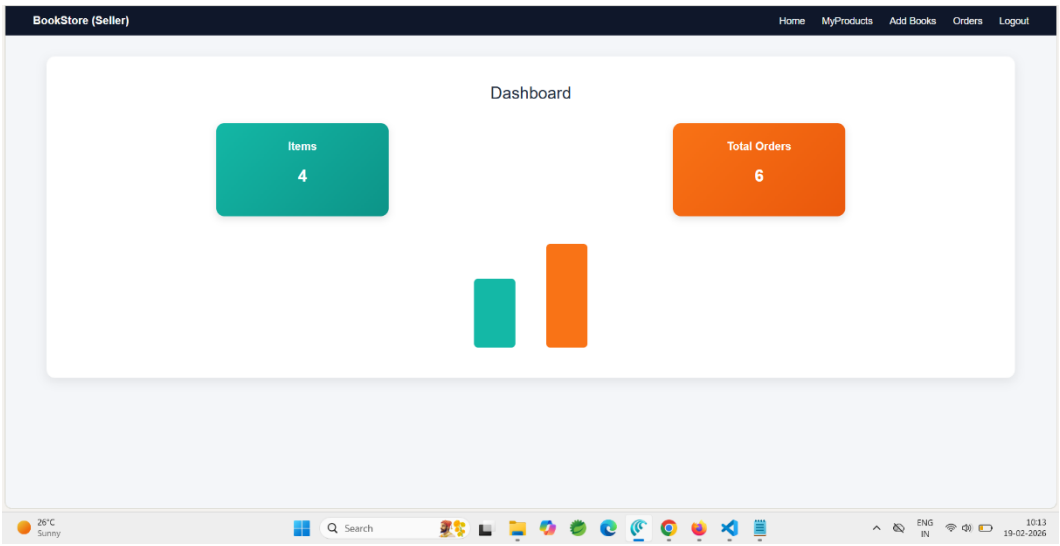

fig: Seller dashboard

10. Testing

Functional testing was performed on all core modules. The team used Postman to ensure that the backend endpoints return the correct status codes and JSON structures. Manual UI testing was conducted on different screen sizes to ensure the layout remains responsive.

11. Known Issues

1. Image Caching: Occasionally, newly added product images may take time to refresh in the grid.
2. Pagination: The catalog current loads all products at once; pagination is pending for larger datasets.

12. Future Enhancements

- Integrated Payment Gateway (Stripe/Razorpay)
- Advanced Search with Autocomplete
- User Reviews and Star Ratings
- Push Notifications for Sales